

Python on the IBMi – Chapter 2



After learning the basic spells of Python, our adventurer now approaches a cave with the word `pyodbc` carved above the entrance. Behind it lies the secret to connecting the realms of SQL, DB2, and the IBMi.

In this chapter, we'll learn to open that gateway establishing a connection to DB2 and querying a file to uncover the data hidden deep within.

With the right incantations, even the oldest tables will reveal their secrets. Grab your torch, this part of the journey dives deeper into the unknown. But with each line of code, we inch closer to the treasure.

Prerequisites

- Please refer to chapter 1 for other prerequisites (<https://www.linkedin.com/pulse/python-ibmi-chapter-1-ricky-thompson-zg7te>)
- Code can be found here: (https://github.com/thomprl/ibmi_python_rpg_adventures)
- `pyodbc` - Python ODBC Library (<https://github.com/mkleehammer/pyodbc/wiki>)
- `tabulate` - Python Library to make a text based table (*optional*)(<https://github.com/astanin/python-tabulate>)

To install pyodbc you can run the following command from a bash terminal

```
python3 -m pip install pyodbc
```

Objective

- Read a IBMi DB2 table using pyodbc from a Python script.

What is pyodbc?

It's a Python library that allows you to connect to ODBC databases like SQL Server, IBM DB2, Oracle, MySQL, PostgreSQL and others. More info can be found on GitHub

(<https://github.com/mkleehammer/pyodbc/wiki>).

Note: this is just a library—you still need an appropriate ODBC driver installed on the target system to connect to the database.

Let's Code!

Using the sample database that are on most systems (**qiws.qcustcdt**) I have a small Python script we will use SQL to query the table and output the column names and values into a pipe delimited format to make it a little easier to read.

In my (**/home/rthompson/python/chapter2**) folder we are going to create a new file called (**pyodbc_read.py**) with the following code.

pyodbc_read.py

```
import pyodbc

# Example of connecting to an IBMi system using pyodbc with
# Username and Password
# myIBMi = "myIBMi"
# username = "username"
# password = "password"
# connection = pyodbc.connect(f'DRIVER={{IBM i Access ODBC
# Driver}};SYSTEM={myIBMi};UID={username};PWD={password}')

connstring = 'DSN=*LOCAL;CommitMode=0;'

try:
    # Establish a connection to the database
    connection = pyodbc.connect(connstring)
    print("Connected to DB2 successfully.")

    # Create a cursor from the connection
    cursor = connection.cursor()
```

```
# Execute a SELECT Query
cursor.execute("select * from qiws.qcustcdt")

# Get column names
column_names = [desc[0] for desc in cursor.description]

# Print header row (pipe-delimited)
print('|'.join(column_names))

# Fetch all rows from the executed query
rows = cursor.fetchall()

for row in rows:
    # Convert each row to a pipe-delimited string
    print('|'.join(str(value) for value in row))

except Exception as e:
    print("Error:", e)

finally:
    # Always close the connection
    connection.close()
```

From VS Code Open a PASE terminal by pressing (**Ctrl+Shift+J**) and selecting PASE and then type the following:

```
cd python/chapter2
python3 pyodbc_read.py
```

The output should look similar to this if everything runs correctly.

```
-bash-5.2$ python3 pyodbc_read.py
Connected to DB2 successfully.
CUSNUM|LSTNAM|INIT|STREET|CITY|STATE|ZIPCOD|CDTLMT|CHGCOD|BALDUE|CDTDUE
938472|Henning |G K|4859 Elm Ave |Dallas|TX|75217|5000|3|37.00|0
839283|Jones   |B D|21B NW 135 St|Clay  |NY|13041|400|1|100.00|0
392859|Vine     |S S|PO Box 79    |Borton|VT|5046|700|1|439.00|0
938485|Johnson |J A|3 Alpine Way |Helen  |GA|30545|9999|2|3987.50|33.50
397267|Tyron    |W E|13 Myrtle Dr  |Hector|NY|14841|1000|1|0|0
389572|Stevens  |K L|208 Snow Pass |Denver|CO|80226|400|1|58.75|1.50
846283|Alison   |J S|787 Lake Dr   |Isle   |MN|56342|5000|3|10.00|0
475938|Doe      |J W|59 Archer Rd  |Sutter|CA|95685|700|2|250.00|100.00
693829|Thomas   |A N|3 Dove Circle|Casper |WY|82609|9999|2|0|0
593029|Williams |E D|485 SE 2 Ave   |Dallas|TX|75218|200|1|25.00|0
192837|Lee      |F L|5963 Oak St   |Hector|NY|14841|700|2|489.50|0.50
583990|Abraham  |M T|392 Mill St   |Isle   |MN|56342|9999|3|500.00|0
-bash-5.2$
```

Bonus Tip

For a bonus you can install a Python library called tabulate

(<https://github.com/astanin/python-tabulate>) to give you a nice little text grid to make it even more readable. Run the following command from the PASE terminal.

```
python3 -m pip install tabulate
```

Now create a new Python script file. I called this one (**pyodbc_read2.py**). It's basically the same as (**pyodbc_read.py**). I removed the lines that printed the column name and row information and replaced with the tabulate function.

pyodbc_read2.py

```
import pyodbc
from tabulate import tabulate

# Example of connecting to an IBMi system using pyodbc with
# Username and Password
# myIBMi = "myIBMi"
# username = "username"
# password = "password"
# connection = pyodbc.connect(f'DRIVER={{IBM i Access ODBC
Driver}};SYSTEM={myIBMi};UID={username};PWD={password}')

connstring = 'DSN=*LOCAL;CommitMode=0;'

try:
```

```
# Establish a connection to the database
connection = pyodbc.connect(connstring)
print("Connected to DB2 successfully.")

# Create a cursor from the connection
cursor = connection.cursor()

# Execute a SELECT Query
cursor.execute("select * from qiws.qcustcdt")

# Get column names
column_names = [desc[0] for desc in cursor.description]

# Fetch all rows from the executed query
rows = cursor.fetchall()

# Print the rows in a text table
print(tabulate(rows, headers=column_names, tablefmt="grid"))

except Exception as e:
    print("Error:", e)

finally:
    # Always close the connection
    connection.close()
```

Run the Python Script

```
python3 pyodbc_read2.py
```

If successfully your results should look similar to this.

```
-bash-5.2$ python3 pyodbc_read2.py
Connected to DB2 successfully.
```

CUSNUM	LSTNAM	INIT	STREET	CITY	STATE	ZIPCOD	CDTLMT	CHGCOD	BALDUE	CDTDUE
938472	Henning	G K	4859 Elm Ave	Dallas	TX	75217	5000	3	37	0
839283	Jones	B D	218 NW 135 St	Clay	NY	13041	400	1	100	0
392859	Vine	S S	PO Box 79	Broton	VT	5046	700	1	439	0
938485	Johnson	J A	3 Alpine Way	Helen	GA	30545	9999	2	3987.5	33.5
397267	Tyron	W E	13 Myrtle Dr	Hector	NY	14841	1000	1	0	0
389572	Stevens	K L	208 Snow Pass	Denver	CO	80226	400	1	58.75	1.5
846283	Alison	J S	787 Lake Dr	Isle	MN	56342	5000	3	10	0
475938	Doe	J W	59 Archer Rd	Sutter	CA	95685	700	2	250	100
693829	Thomas	A N	3 Dove Circle	Casper	WY	82609	9999	2	0	0
593029	Williams	E D	485 SE 2 Ave	Dallas	TX	75218	200	1	25	0
192837	Lee	F L	5963 Oak St	Hector	NY	14841	700	2	489.5	0.5
583990	Abraham	M T	392 Mill St	Isle	MN	56342	9999	3	500	0

```
-bash-5.2$
```

What's Next?

With this challenge behind him, our hero presses onward, only to face a great divide: the daunting task of calling an RPG program from Python using pyodbc. The path forward isn't clear, and the canyon between the two worlds seems impossible to cross.

Will our brave adventurer find a way to bridge the gap?

Tune in next week to see if he can swing across and unlock the next chapter of this quest.